

Prototipado nativo

Gestión de la
calidad del software

Contenidos

- Definiciones
- Ventajas e inconvenientes
- Plataformas y prototipos nativos
- Aplicaciones nativas/híbridas

Definiciones

- Prototipo:
 - Un prototipo es una modelización interactiva y funcional que permite simular el comportamiento de una aplicación antes de su desarrollo completo.
- Los prototipos se pueden clasificar según su grado de fidelidad:
 - Baja fidelidad (lo-fi): bocetos en papel o wireframes estáticos. Son rápidos y baratos, ideales para las primeras fases.
 - Media fidelidad: mockups con interacción básica, sin datos reales ni lógica de negocio.
 - Alta fidelidad (hi-fi): típicamente un prototipo nativo, que se aproxima al máximo al producto final tanto en aspecto como en comportamiento.

Definiciones

- Se suele distinguir entre prototipos desechables (throwaway), que se tiran una vez validados, y prototipos evolutivos, donde el código del prototipo evoluciona hacia el producto final.
 - El prototipado nativo está especialmente vinculado al modelo evolutivo, aunque no siempre.
- Prototipo nativo, definición:
 - Es un prototipo con funcionalidad en la capa del interfaz (al menos), que ha sido desarrollado con la misma tecnología con la que se implementará el producto final.

Ventajas

- Fidelidad del interfaz, el prototipo será muy cercano al aspecto que tendrá el producto final.
- Mejora de la UX: reduce curva aprendizaje, el usuario está habiturado al interfaz.
- Validación de funcionalidades y alineación con requisitos de usuarios e inversores.
- Facilitar la participación de las partes interesadas

Ventajas

- Testing sobre el prototipo nativo: pruebas de usabilidad, A/B testing, etc.
 - El prototipo nativo permite hacer estos tests de forma mucho más realista que los prototipos lo-fi.
- Al trabajar con la tecnología final obtenemos:
 - Reducción de costes y tiempos
 - Detección temprana de problemas
 - Sistema probado en plataformas reales

Inconvenientes

- Mayor complejidad en el desarrollo: trabajar con tecnologías finales requiere más conocimientos, recursos, equipos, etc.
- Incremento de costes: tiempo y recursos (humanos, SW, HW).
- **Código no reutilizable:** si se decide cambiar de plataforma o tecnología.
- Mantenimiento y actualizaciones: cambios de versiones en el uso de SO, bibliotecas y componentes de 3as partes.
- Cobertura multiplataforma limitada: un prototipo nativo Android no sirve para validar en iOS y viceversa.
- Riesgo de scope creep: al tener un prototipo tan funcional, los stakeholders pueden empezar a pedir funcionalidades que van más allá de lo presupuestado

Cuando usar un prototipo nativo

- No siempre el prototipo nativo es la mejor opción.
- Un prototipo lo-fi puede ser suficiente para validar un flujo de navegación, mientras que el prototipo nativo es imprescindible cuando lo que se quiere probar no puede simularse en papel ni en una herramienta de diseño.
- Factores a tener en cuenta:
 - El producto requiere acceso a hardware del dispositivo: cámara, GPS, etc.
 - El rendimiento y la fluidez de la UI son críticos para el negocio (apps de realidad aumentada, videojuegos, edición de vídeo).
 - Es necesario realizar pruebas de sobre dispositivos reales con usuarios finales
 - El equipo ya domina la tecnología final y el sobrecoste del prototipo nativo respecto a un lo-fi es bajo.

Cuando usar un prototipo nativo

- No es buena idea un prototipo nativo si:
 - El objetivo es únicamente validar la arquitectura de la información o el flujo de pantallas: un wireframe interactivo es suficiente y mucho más rápido.
 - El equipo no tiene experiencia en la tecnología nativa objetivo
 - El presupuesto y el tiempo son muy limitados
 - La plataforma destino no está definida todavía: no se sabe si el producto será Android, iOS, web o multiplataforma.
- El prototipo nativo aporta mayor valor cuando el riesgo es de naturaleza técnica o de experiencia real de uso, no cuando es conceptual o de negocio.
- Regla práctica es: usar el prototipo de menor fidelidad que sea suficiente para responder la pregunta que se quiere validar.

Plataformas

- PCs (sobremesa, portátiles)
 - Sistemas operativos: Windows, Linux, OSX
 - Tecnologías: .net, WPF, UWP, WinUI 3
 - Lenguajes: C#, C++, Java, Python, Swift
 - Frameworks: .NET MAUI, Electron, SwiftUI

Plataformas

- Web
 - Tecnologías nativas web: HTML5, CSS, Javascript
 - Frameworks Front-end: Angular, Vue, React,
 - Frameworks Full-stack: Django (Python), Next.js (React + Node), Nuxt (Vue + Node), SpringBoot (Java), Laravel (PHP)

Plataformas

- Móviles, tablets, etc.
 - Sistemas operativos: Android, iOS
 - Frameworks/tecnologías: Android Studio, Swift
 - Flutter (Dart)
 - React Native (JavaScript/TypeScript):
 - Ionic (HTML/CSS/JS)
 - Kotlin Multiplatform (KMP)
 - Swift/SwiftUI: estándar para iOS/macOS.
 - Lenguajes: Java, Kotlin, Dart, JavaScript, TypeScript, XCode

Frameworks de desarrollo para móviles

Framework	Tecnología	Rendimiento	Acceso a HW	Curva aprendizaje	Ejemplos de uso
Android Studio	Kotlin, Java	Máximo APIs del SO	Completo APIs del SO	Alta IDE complejo	Google, Spotify, Netflix
SwiftUI	Swift	Máximo APIs del SO	Completo APIs del SO	Alta Ecosistema cerrado	Apple, Instagram, LinkedIn
Flutter	Dart	Muy Alto Compila a código ARM	Alto, vía plugins	Media No muy conocido	Google, eBay, BMW
React Native	Javascript, Typescript	Alto Puente JS/Nativo	Alto, componentes nativos y plugins	Media (baja si conoces react)	Meta, discord
Ionic	HTML/CSS/JS Angular/react/vue	Medio (usa webviews)	Medio (usa Capacitor)	Baja (si sabes tecnologías web)	Nationwide, marketWatch
.NET MAUI	.net, C#	Alto Compila a nativo	Alto (APIs nativas)	Media (si sabes .net y C#)	Microsoft, UPS

Aplicaciones nativas/híbridas

- Para desarrollar aplicaciones para móviles, tablets, etc. tenemos tres opciones en cuanto a su arquitectura:
 - Página web adaptada a las pantallas de móviles/tablets (PWA)
 - Aplicación híbrida: Ionic, React Native
 - Aplicación nativa: Android Studio

Aplicaciones nativas/híbridas

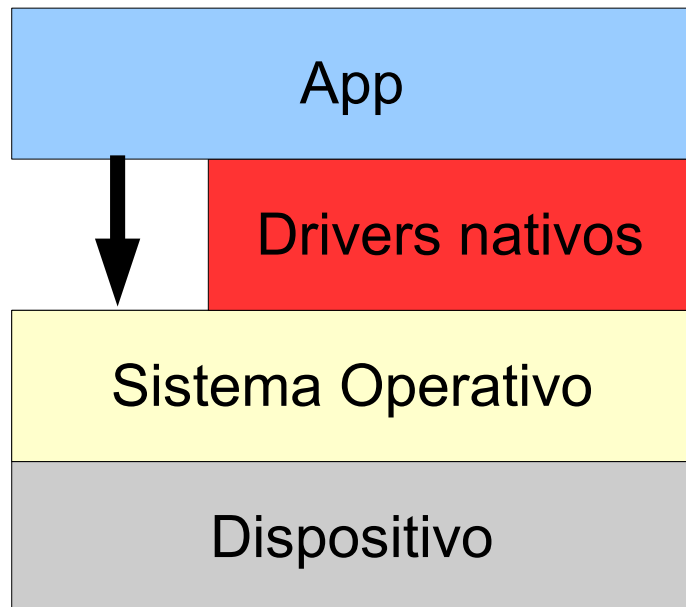
- Página web (PWA)



- Accede a algunas APIs nativas del dispositivo (cámara, geolocalización, notificaciones push, almacenamiento offline) mediante las Web APIs estándar.
- Limitaciones: en iOS el soporte de PWA es inferior al de Android.
- Ventajas: un solo código, actualización instantánea.

Aplicaciones nativas/híbridas

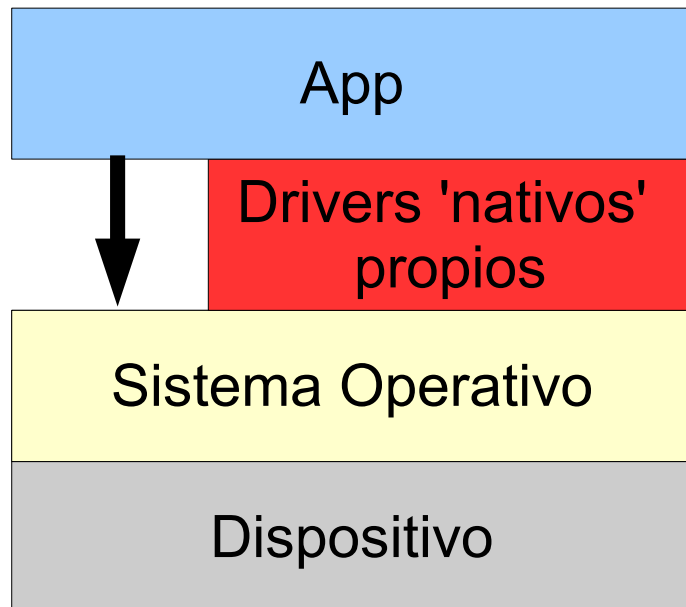
- App nativa



- Android: Kotlin + Jetpack Compose (UI declarativa moderna) o Java + XML (legacy).
- iOS: Swift + SwiftUI (UI declarativa moderna) o Objective-C + UIKit (legacy).
- Máximo rendimiento, acceso completo a todas las APIs del SO, mejor integración con el look & feel de la plataforma.
- Coste: hay que mantener dos versiones de código (Android e iOS).

Aplicaciones nativas/híbridas

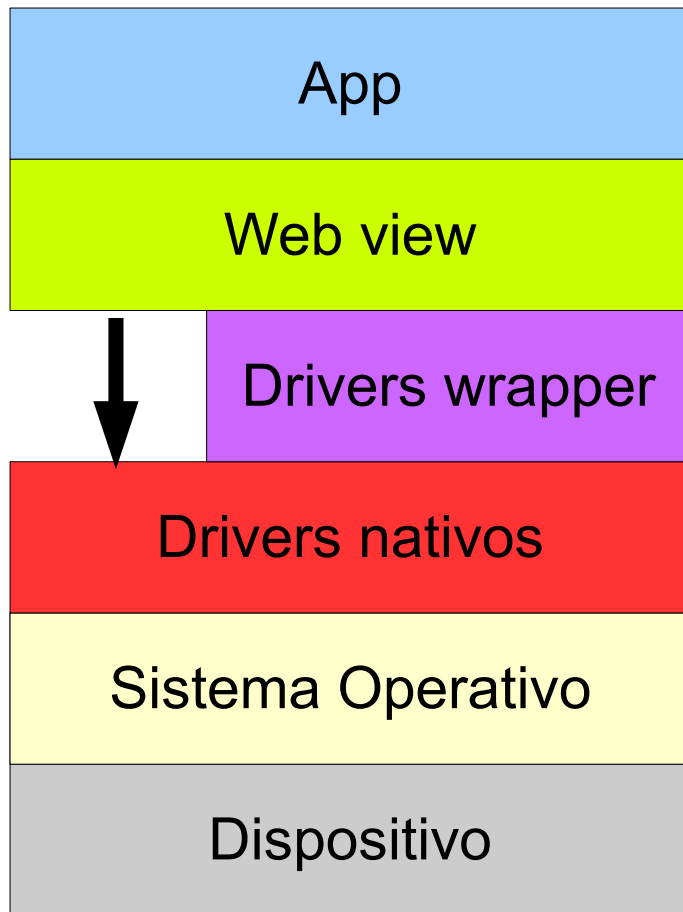
- App nativa 'compilada'



- Flutter, .net MAUI
- El código se compila (no interpreta) a código nativo ARM.
- Flutter tiene su propio motor de renderizado, por lo que la UI no es del SO pero el rendimiento es casi nativo.
- .NET MAUI usa controles nativos de cada plataforma desde un código C# compartido.
- Se sitúa entre "híbrida" y "nativa pura" en cuanto a capacidades y rendimiento.

Aplicaciones nativas/híbridas

- App híbrida



- El código web corre dentro de una WebView nativa. Un wrapper (Capacitor/Cordova) expone APIs nativas al código JavaScript.
- El aspecto imita al nativo mediante componentes de UI de la librería (ej. Ionic components), pero no son los componentes del SO.
- Rendimiento inferior para animaciones complejas o gráficos intensivos.
- **React Native, caso particular:** React Native no usa WebView: los componentes JS se traducen en tiempo de ejecución a componentes nativos del SO (UIView en iOS, View en Android).

Tendencias actuales

- Apps nativas: Jetpack Compose de Android Studio y Swift + SwiftUI
 - Mención especial a Kotlin Multiplatform, que permite desarrollo multiplataforma con UI nativa
- Apps híbridas: Flutter se está consolidando como primera opción multiplataforma, superando a React Native.

Tendencias actuales

- El uso de herramientas asistidas por IA para el prototipado que reducen el tiempo de desarrollo de prototipos nativos.
 - **Herramientas de generación de UI:** plataformas que generan interfaces completas a partir de una descripción en lenguaje natural.
 - **Asistentes de código:** GitHub Copilot, Cursor o Claude aceleran la escritura de código nativo, reduciendo el tiempo de construcción de un prototipo nativo de días a horas.
 - **Diseño a código:** herramientas que convierten diseños de Figma (Lo-Fi) directamente en código React Native o Flutter, eliminando el trabajo manual de implementación del prototipo.